DEEP LEARNING

Chapter 6 Quiz Bank

Concepts • True/False • Worked Mathematics • Graph Plotting

KL Divergence • Cross-Entropy • Logistic Regression • SGD • Feedforward Networks • XOR • ReLU • Sigmoid/Softmax • Architecture • Backpropagation

SECTION A —short conceptual questions and answers

A1. What does entropy measure in a target distribution?

Answer: Entropy H(P) measures uncertainty in the target distribution itself. For P = (0.7, 0.3), H(P) = -[0.7 ln(0.7) + 0.3 ln(0.3)] = 0.6109 nats.

A2. How are cross-entropy and KL divergence different?

Answer: Cross-entropy H(P,Q) is the average information cost when the model distribution Q represents outcomes from target P. KL divergence D_KL(P||Q) is the extra information cost caused by the mismatch between P and Q.

A3. State the key relationship between entropy, cross-entropy, and KL divergence.

Answer: **H(P,Q) = H(P) + D_KL(P||Q).** The target entropy is fixed when P is fixed, so minimizing cross-entropy also minimizes the KL divergence.

A4. What happens to KL divergence when the predicted distribution equals the target distribution?

Answer: If Q = P, then D_KL(P||Q) = 0. Cross-entropy then equals the target entropy H(P); the entropy does not disappear.

A5. Why is logistic regression used for classification despite its name?

Answer: It calculates a linear score $z = w^T x + b$ and applies sigmoid to estimate P(y=1|x). That probability can be thresholded to predict one of two classes.

A6. What does the sigmoid function do in logistic regression?

Answer: σ(z) = 1/(1+e^(-z)) **transforms a real-valued score into a probability strictly between zero and one** for finite z.

A7. What does binary cross-entropy represent?

Answer: L(y,p) = -[y ln(p) + (1-y) ln(1-p)] is the negative log-likelihood of a Bernoulli output distribution. The training objective is to minimize it.

A8. What information does a gradient provide?

Answer: A gradient describes how the loss changes with each trainable parameter. Its sign and magnitude indicate the local direction and sensitivity of the loss to parameter changes.

A9. How does SGD use a gradient?

Answer: SGD updates parameters using θ_new = θ_old - η$\hat{g}$, where $\hat{g}$ is an estimated gradient and η is the learning rate. It moves in the direction opposite to the estimated gradient.

A10. What is the difference between forward propagation and backpropagation?

Answer: Forward propagation computes hidden activations, logits, output probabilities, and loss from input to output. Backpropagation starts from the loss derivative and propagates gradients backward through the network.

A11. What does a basic neuron compute?

Answer: It forms a weighted sum plus a bias, $z = w^T x + b$, then applies an activation $a = f(z)$. Weights control feature contributions; the bias shifts the pre-activation.

A12. What is a feedforward network?

Answer: It processes information from input to output without recurrent loops. A simple network transforms x into hidden representation h and then into output y.

A13. Why can a single perceptron not learn XOR?

Answer: A single perceptron has a straight-line decision boundary for two inputs. XOR's positive and negative labels alternate across the four corners, so no one straight line separates them.

A14. How can a hidden layer represent XOR?

Answer: It defines h1 = x1 OR x2 and h2 = x1 AND x2, then y = h1 AND NOT(h2). The hidden units create an intermediate representation that makes the final decision possible.

A15. Why are nonlinear activation functions necessary?

Answer: A nonlinear activation such as ReLU allows the network to represent nonlinear relationships.

A16. Why is ReLU commonly used for hidden units?

Answer: ReLU(x) = max(0,x) is simple and computationally inexpensive. For positive inputs its derivative is one, so it avoids the positive-side saturation associated with sigmoid and tanh.

A17. What is a limitation of ReLU?

Answer: For a negative pre-activation, ReLU outputs zero and its derivative is zero. If a unit stays in that region for all its inputs, it may stop updating through that path; this is called the dying ReLU problem.

A18. When is sigmoid appropriate, and when is softmax appropriate?

Answer: Sigmoid represents a Bernoulli output for a binary target. Softmax represents a probability distribution over mutually exclusive classes, with all class probabilities summing to one.

A19. How are gradient, Jacobian, backpropagation, and an optimizer related?

Answer: A gradient is for a scalar loss; a Jacobian records how vector outputs change with vector inputs. Backpropagation applies the chain rule and local Jacobians efficiently to calculate gradients; SGD or another optimizer then updates parameters.

SECTION B — true/false questions with reasons

B1. KL divergence is always symmetric: D_KL(P||Q) = D_KL(Q||P).

FALSE — exchanging P and Q generally changes the value.

B2. KL divergence is non-negative and equals zero when P = Q.

TRUE — Matching target and model distributions removes the mismatch.

B3. For fixed target P, minimizing cross-entropy is equivalent to minimizing KL divergence.

TRUE — H(P,Q) = H(P) + D_KL(P||Q), and H(P) is constant with respect to the model parameters.

B4. The entropy of a one-hot target distribution is zero.

TRUE — A one-hot target assigns probability one to the true class and zero to the others.

B5. Logistic regression uses a linear score followed by sigmoid to estimate a binary-class probability.

TRUE — as $p = \sigma(w^T x+b)$.

B6. For every finite logit z, sigmoid outputs exactly 0 or exactly 1.

FALSE — For finite z, sigmoid lies strictly between zero and one.

B7. When the true label is y=1, binary cross-entropy simplifies to -ln(p).

TRUE — Substituting y=1 into the binary cross-entropy formula leaves -ln(p).

B8. ReLU returns zero for a negative input and passes a positive input through unchanged.

TRUE — ReLU(x)=max(0,x).

B9. XOR is linearly separable, so a single perceptron can learn it.

FALSE — XOR is not linearly separable; a single straight boundary cannot separate its classes.

B10. Softmax is the natural complete output distribution when multiple mutually exclusive classes are possible.

TRUE — It returns one probability per class and the probabilities sum to one.

B11. A single sigmoid output and a three-class Softmax output have identical output structures.

FALSE — Sigmoid gives one positive-class probability; Softmax gives one probability for each mutually exclusive class.

B12. The fully connected 2–2–3 MLP has 15 trainable parameters.

TRUE — There are 4 input-to-hidden weights + 2 hidden biases + 6 hidden-to-output weights + 3 output biases = 15.

B13. The universal approximation theorem guarantees that gradient descent will find the exact desired parameters.

FALSE — The theorem concerns representational capability; it does not guarantee that a training algorithm finds the right parameters.

B14. It claims that deeper networks are always better and easier to optimize.

FALSE — Depth and width must be selected and evaluated; deeper networks are not always better or easier to optimize.

B15. Backpropagation itself is the parameter-update optimizer.

FALSE — Backpropagation calculates gradients; SGD or another optimizer uses them to update parameters.

B16. If a hidden ReLU pre-activation is negative, its local ReLU derivative is zero and that gradient path is blocked for the current input.

TRUE — it gives $\mathrm{ReLU}'(z)=0$ for $z<0$.

SECTION C — mathematical problems with worked solutions

C1. Calculate the entropy of the target distribution P=(0.7, 0.3).

Formula: $H(P) = -\Sigma_i P(i) \ln P(i)$.

Substitute: $H(P) = -[0.7 \ln(0.7) + 0.3 \ln(0.3)]$.

Using $\ln(0.7) \approx -0.3567$ and $\ln(0.3) \approx -1.2040$: $H(P) \approx 0.2497 + 0.3612 = 0.6109$ nats.

Answer: 0.6109 nats.

C2. For target P=(0.7,0.3) and model Q=(0.6,0.4), calculate cross-entropy.

Formula: $H(P,Q) = -\Sigma_i P(i) \ln Q(i)$.

$H(P,Q) = -[0.7 \ln(0.6) + 0.3 \ln(0.4)]$.

Using $\ln(0.6) \approx -0.5108$ and $\ln(0.4) \approx -0.9163$: $H(P,Q) \approx 0.3576 + 0.2749 = 0.6325$ nats.

Answer: 0.6325 nats.

C3. Using P=(0.7,0.3) and Q=(0.6,0.4), calculate KL divergence and verify the identity.

$D_{KL}(P||Q) = \Sigma_i P(i) \ln[P(i)/Q(i)]$.

$D_{KL} = 0.7 \ln(0.7/0.6) + 0.3 \ln(0.3/0.4) \approx 0.1079 - 0.0863 = 0.0216$ nats.

C4. A one-hot target has true class A. Compare the cross-entropy when the model assigns p=0.70 and p=0.95 to class A.

For a one-hot target, $L = -\ln(p)$ for the true class.

At p=0.70: $L = -\ln(0.70) \approx 0.3567$ nats.

At p=0.95: $L = -\ln(0.95) \approx 0.0513$ nats.

The second loss is smaller because the model assigns a higher probability to the correct class.

C5. Logistic regression receives x=2 with w=0 and b=0. Calculate z and p.

Linear score: $z=wx+b=(0)(2)+0=0$.

Sigmoid: $p=1/(1+e^{(-z)})=1/(1+e^{0})=1/2=0.5$.

Answer: z=0 and p=0.50, an uncertain binary prediction.

C6. For a true malicious label y=1, compare binary cross-entropy for p=0.80 and p=0.10.

When y=1, $L=-\ln(p)$.

p=0.80: $L=-\ln(0.80)\approx0.2231$ nats.

p=0.10: $L=-\ln(0.10)\approx2.3026$ nats.

The confidently wrong prediction p=0.10 receives the much larger loss.

C7. For x=2, y=1, w=0, b=0, p=0.5, calculate the gradients and one SGD update with η=0.1.

For sigmoid + binary cross-entropy: $\partial L/\partial z=p-y$, $\partial L/\partial w=(p-y)x$, $\partial L/\partial b=p-y$.

Weight gradient: $(0.5-1)(2)=-1$. Bias gradient: $0.5-1=-0.5$.

$w_{new} = 0 - 0.1(-1)=0.10$. $b_{new} = 0 - 0.1(-0.5)=0.05$.

Answer: gradients are -1 and -0.5; updated parameters are w=0.10 and b=0.05.

C8. Continue C7. Calculate the updated prediction and loss.

$z_{new} = w_{new} x + b_{new} = (0.1)(2)+0.05=0.25$.

$p_{new}=\sigma(0.25)=1/(1+e^{(-0.25)})\approx0.5622$.

Because y=1, $L_{new}=-\ln(0.5622)\approx0.5759$ nats.

Comparison: loss falls from 0.6931 to 0.5759 nats; attack probability rises from 50.00% to about 56.22%.

C9. A neuron has $x_1=2$, $x_2=3$, $w_1=0.5$, $w_2=0.2$, $b=1$. Find its pre-activation z.

$z=w_1x_1+w_2x_2+b$.

$z=(0.5)(2)+(0.2)(3)+1=1+0.6+1=2.6$.

Answer: $z=2.6$.

C10. Calculate ReLU(-3), ReLU(0), and ReLU(2.7).

$\mathrm{ReLU}(x)=\max(0,x)$.

$\mathrm{ReLU}(-3)=0$; $\mathrm{ReLU}(0)=0$; $\mathrm{ReLU}(2.7)=2.7$.

Rule: negative inputs become zero; positive inputs pass through unchanged.

C11. For $x_1=2$ and $x_2=1$, calculate the two hidden scores $z_1=x_1+x_2-1$ and $z_2=x_1-x_2$, then apply ReLU.

$z_1=2+1-1=2$. $z_2=2-1=1$.

$h_1=\mathrm{ReLU}(2)=2$; $h_2=\mathrm{ReLU}(1)=1$.

Answer: hidden representation $h=[2,1]^T$.

C12. A binary classifier has logit $z=1$ and true label $y=1$. Calculate p and loss.

$p=\sigma(1)=1/(1+e^{(-1)})\approx0.7311$.

$L=-\ln(0.7311)\approx0.3133$ nats.

Answer: $p\approx0.7311$ and $L\approx0.3133$ nats.

C13. Calculate the Softmax probabilities for logits [0,2,1] for Benign, DDoS, and Port scan.

Exponentials: $e^0=1$, $e^2\approx7.389$, $e^1\approx2.718$; sum≈11.107.

P(Benign)$=1/11.107\approx0.090$; P(DDoS)$=7.389/11.107\approx0.665$; P(Port scan)$=2.718/11.107\approx0.245$.

The probabilities sum to approximately 1.000. The largest is DDoS.

C14. For C13, if the correct class is DDoS with one-hot target $y=[0,1,0]$, calculate cross-entropy and the logit gradient.

Loss: $L=-\ln(0.665241)\approx0.407606$ nats (about 0.408).

Softmax + cross-entropy gives $\delta=p-y$.

$\delta\approx[0.090031, 0.665241-1, 0.244728]^T = [0.090031, -0.334759, 0.244728]^T$.

C15. Count trainable parameters in a fully connected 2–2–3 MLP with biases.

Input-to-hidden: $2\times2=4$ weights + 2 hidden biases = 6 parameters.

Hidden-to-output: $2\times3=6$ weights + 3 output biases = 9 parameters.

Total = $6+9=15$ trainable parameters.

C16. Let $u=2x$ and $L=u^2$. Use the chain rule to calculate dL/dx at $x=3$.

At $x=3$, $u=2(3)=6$.

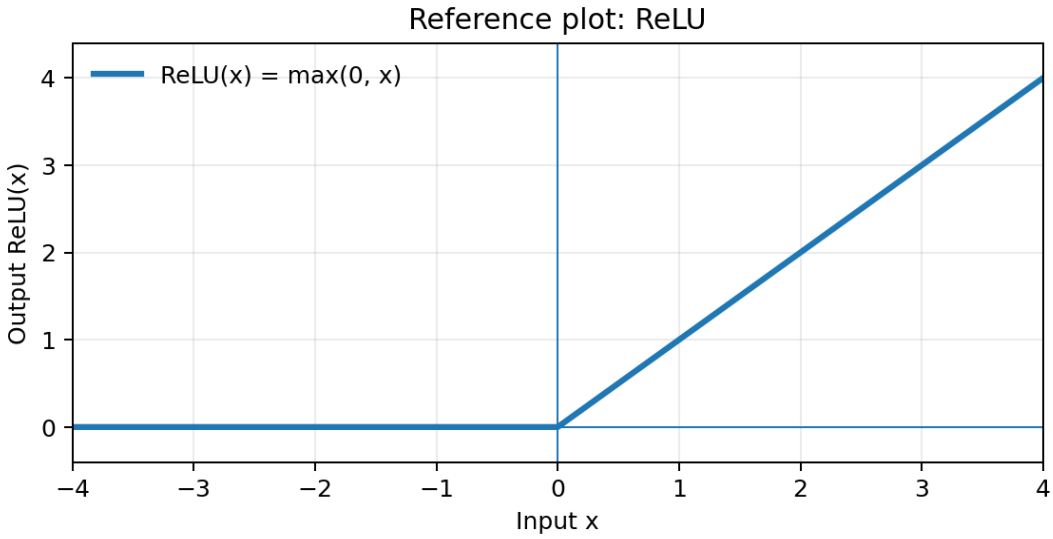
$dL/du=2u=12$ and $du/dx=2$.

$dL/dx=(dL/du)(du/dx)=12\times2=24$.

SECTION D — graph-plotting exercises with reference answers

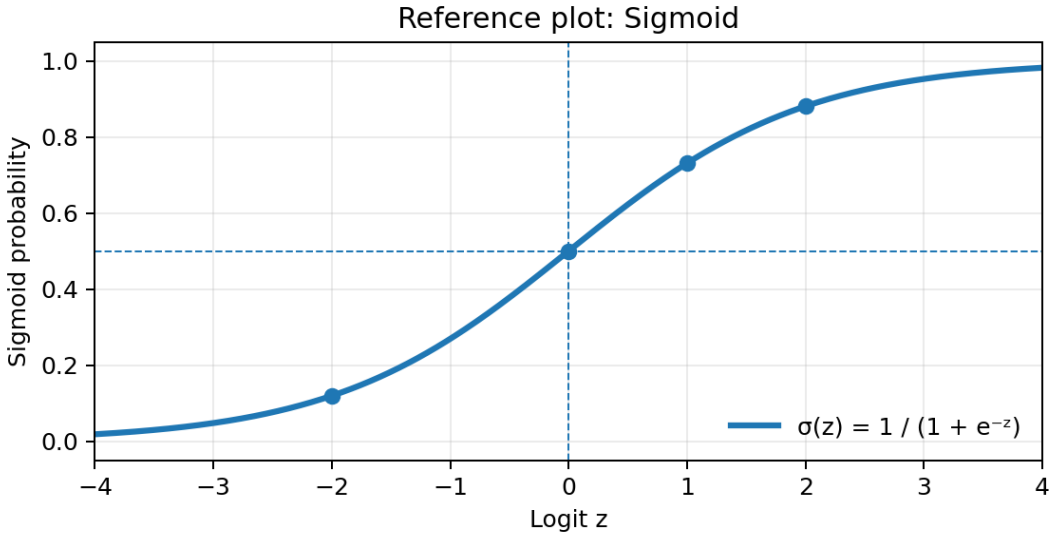
D1. Plot the ReLU activation function.

Task: Plot y = ReLU(x) = max(0,x) for x from -4 to 4. Mark at least (-4,0), (-2,0), (0,0), (1,1), (2,2), and (4,4).
Expected result: The graph is flat at y=0 for x<0 and follows y=x for x≥0.
PDF basis: ReLU(x) = max(0,x).



D2. Plot the sigmoid function.

Task: Plot σ(z)=1/(1+e^(-z)). Use the PDF values z=-2→0.1192, z=0→0.5000, z=1→0.7311, and z=2→0.8808.
Expected result: Draw a smooth S-shaped curve between 0 and 1. The output is 0.5 when z=0.
PDF basis: Sigmoid converts a real-valued logit into a probability strictly between 0 and 1 for finite z.

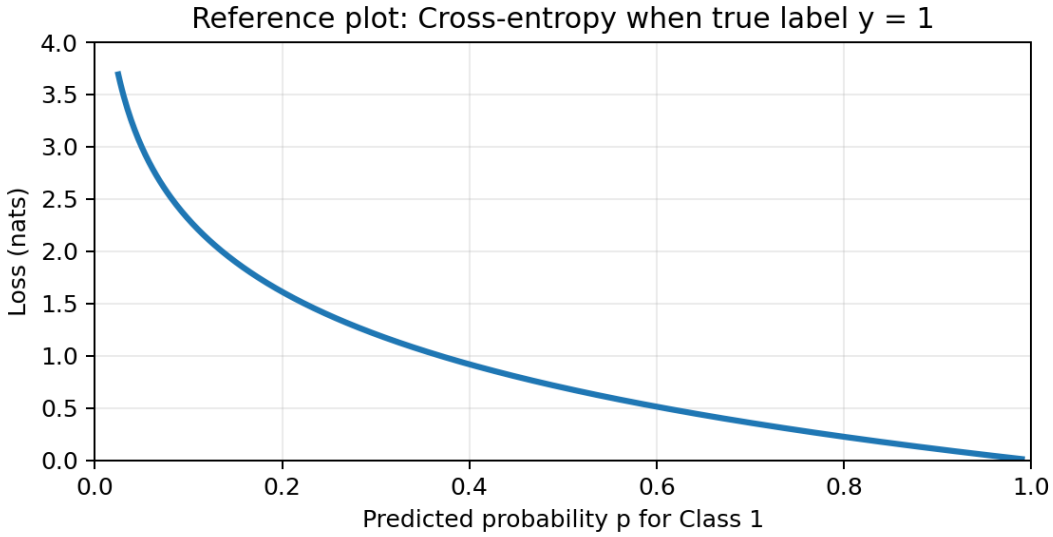


D3. Plot binary cross-entropy when the true label is y=1.

Task: Plot L(p)=-ln(p). Suggested points: (0.1,2.3026), (0.2,1.6094), (0.3,1.2040), (0.4,0.9163), (0.5,0.6931), (0.6,0.5108), (0.7,0.3567), (0.8,0.2231), (0.9,0.1053).

Expected result: The curve decreases as p increases. A probability near 1 gives a small loss; a probability near 0 gives a large loss.

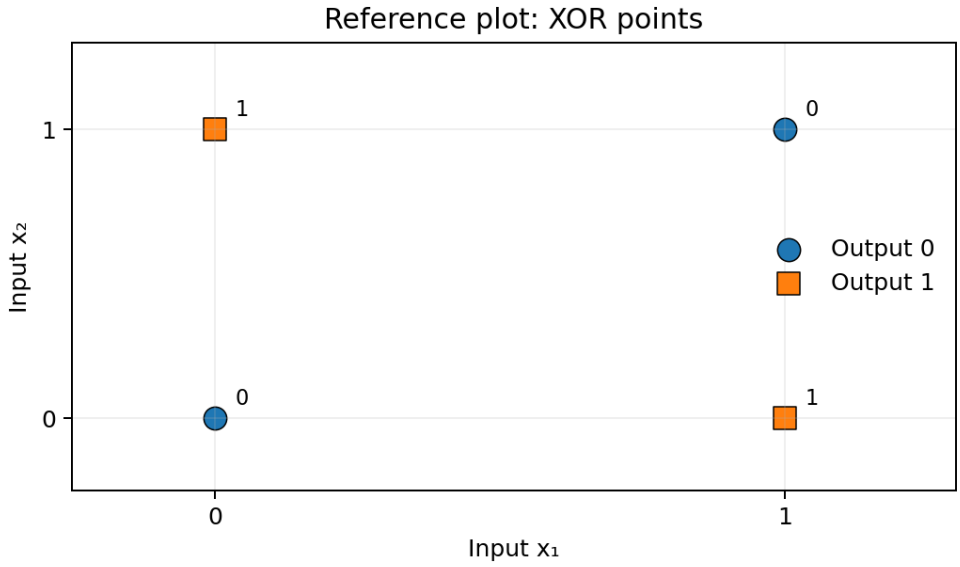
PDF basis: For a true label y=1, L(p)=-ln(p).



D4. Plot the four XOR input points.

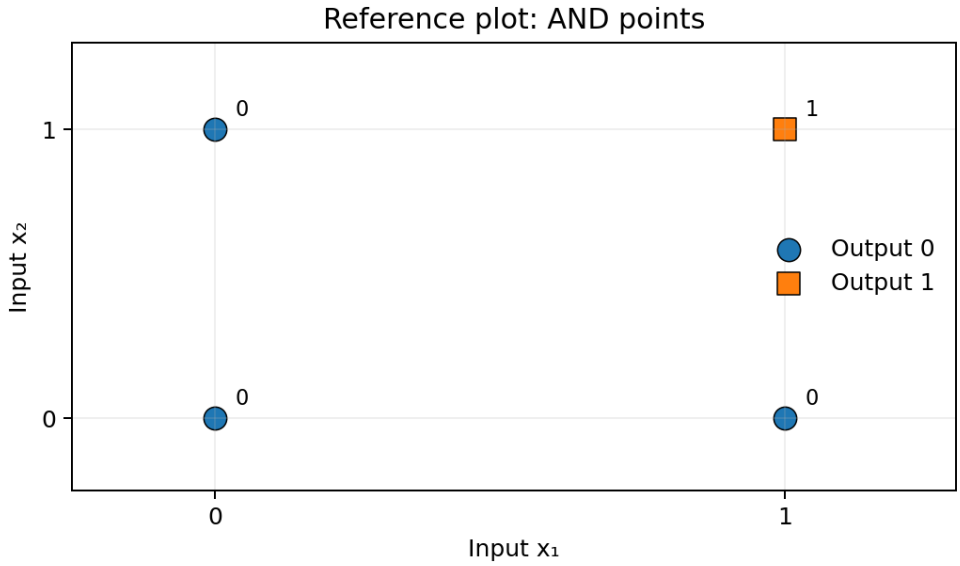
Task: Plot (x_1,x_2) and label the output: (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0. Use different symbols for output 0 and output 1.

Expected result: The labels alternate at the corners. No single straight line separates the two positive points from the two negative points.



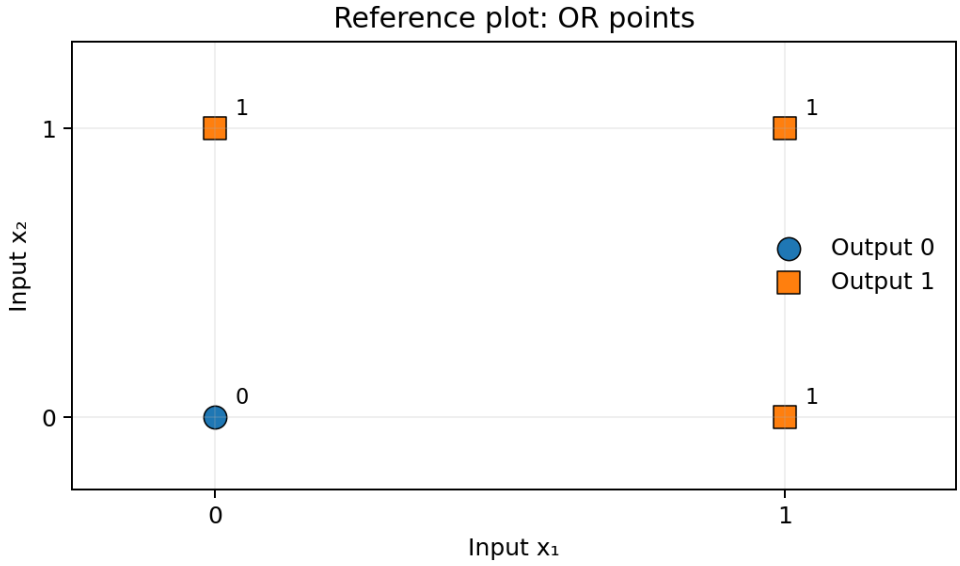
D5. Plot the AND truth table as points.

Task: Plot (0,0)→0, (0,1)→0, (1,0)→0, (1,1)→1.
Expected result: Only the point (1,1) has output 1; the classes are linearly separable.
PDF basis: AND is linearly separable.



D6. Plot the OR truth table as points.

Task: Plot (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→1.
Expected result: Only (0,0) has output 0; the classes are linearly separable.
PDF basis: OR is linearly separable.



Quick formula sheet from the PDF

Concept	Formula / rule
Entropy	$H(P) = -\Sigma_i P(i) \ln P(i)$
Cross-entropy	$H(P,Q) = -\Sigma_i P(i) \ln Q(i)$
KL divergence	$D_{KL}(P\|\|Q) = \Sigma_i P(i) \ln[P(i)/Q(i)]$
Relationship	$H(P,Q) = H(P) + D_{KL}(P\|\|Q)$
Logistic score	$z = w^T x + b$
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$
Binary cross-entropy	$L(y,p) = -[y \ln(p) + (1-y) \ln(1-p)]$
Gradient for sigmoid + BCE	$\partial L/\partial z = p-y$; $\partial L/\partial w = (p-y)x$; $\partial L/\partial b = p-y$
SGD update	$\theta_{new} = \theta_{old} - \eta \hat{g}$
ReLU	$ReLU(z) = \max(0,z)$
Softmax	$p_k = e^{z_k} / \Sigma_j e^{z_j}$
Softmax + cross-entropy gradient	$\partial L/\partial z_k = p_k - y_k$
Chain rule	$dL/dx = (dL/du)(du/dx)$
Vector chain rule	$\nabla_x L = J_f(x)^T \nabla_y L$